

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

_____ oo0oo _____



**BÀI TẬP LỚN: KỸ THUẬT RA QUYẾT
ĐỊNH**

**ĐỀ TÀI: WEB MÔ PHỎNG TRỰC QUAN
THUẬT TOÁN DIJKSTRA TRÊN ĐỒ THỊ**

Giảng viên hướng dẫn: ThS. Hồ Thanh Phương

Sinh viên thực hiện: Nguyễn Duy Tuyên

MSSV: 2213821

Lớp: L02

TP. Hồ Chí Minh, tháng 05 năm 2026

Mục lục

TÓM TẮT	1
1 GIỚI THIỆU	2
1.1 Lý do chọn đề tài	2
1.2 Mục tiêu đề tài	2
2 CƠ SỞ LÝ THUYẾT	3
2.1 Bản chất của thuật toán Dijkstra	3
2.2 Mã giả (Pseudocode) và Quy trình gán nhãn	3
2.3 Lưu đồ giải thuật	4
2.4 Đánh giá độ phức tạp (Complexity Analysis)	5
3 THIẾT KẾ VÀ HIỆN THỰC	6
3.1 Lựa chọn công nghệ	6
3.2 Cấu trúc dữ liệu và Vận hành ứng dụng	6
4 KẾT QUẢ VÀ THẢO LUẬN	8
4.1 Trải nghiệm người dùng trên Dashboard	8
4.2 Đánh giá tính hiệu quả	9
4.3 Xử lý các trường hợp ngoại lệ (Edge Cases)	10
5 KẾT LUẬN	11
TÀI LIỆU THAM KHẢO	12
PHỤ LỤC	13

Danh mục hình ảnh

1	Lưu đồ các bước thực hiện thuật toán trong ứng dụng	4
2	Mô hình kiến trúc phân lớp của simulator	6
3	Trình tự xử lý dữ liệu từ lúc bấm nút đến lúc hiện kết quả	7
4	Toàn cảnh giao diện mô phỏng với đồ thị mẫu	8
5	Phân tích kết quả và bảng nhật ký giải thuật	9
6	Phân tích truy vết lộ trình hiển thị trên hệ thống	10

TÓM TẮT

Báo cáo trình bày quá trình thiết kế và hiện thực ứng dụng web mô phỏng trực quan thuật toán Dijkstra - một trong những giải pháp kinh điển nhất trong lý thuyết đồ thị để tìm đường đi ngắn nhất. Nhằm giải quyết khó khăn của sinh viên khi tiếp cận thuật toán qua các con số khô khan trên giấy, ứng dụng cung cấp một môi trường tương tác (simulator) cho phép người dùng tự do xây dựng đồ thị, tùy chỉnh trọng số và quan sát trực tiếp quá trình lan truyền của thuật toán. Thông qua sự kết hợp giữa giao diện thân thiện (HTML/CSS), thư viện vẽ đồ thị động (Vis.js) và logic xử lý nội tại (JavaScript), kết quả đạt được không chỉ là một công cụ tính toán chính xác mà còn là một nền tảng giáo dục trực quan, giúp người học đối chiếu và hiểu rõ bản chất cốt lõi của thuật toán.

1 GIỚI THIỆU

1.1 Lý do chọn đề tài

Trong kỷ nguyên số hóa hiện nay, tối ưu hóa đường đi là một trong những vấn đề cốt lõi trong nhiều lĩnh vực như giao thông vận tải, logistics, mạng máy tính và trí tuệ nhân tạo. Thuật toán Dijkstra, được Edsger W. Dijkstra đề xuất năm 1959, đã trở thành giải pháp kinh điển và là tiêu chuẩn vàng để tìm đường đi ngắn nhất trong đồ thị có trọng số không âm.

Tuy nhiên, đối với sinh viên mới làm quen, thuật toán Dijkstra thường được tiếp cận qua các công thức toán học khô khan và bảng gán nhãn trừu tượng. Việc tính toán thủ công trên giấy với đồ thị phức tạp dễ khiến người học bị mất phương hướng, khó nắm bắt được bản chất “lan tỏa” (breadth-first search with priority) của thuật toán.

Chính vì vậy, em đã chọn phát triển một **ứng dụng web mô phỏng trực quan thuật toán Dijkstra** nhằm biến những khái niệm trừu tượng thành các bước minh họa sinh động, giúp người học dễ dàng tiếp thu, hiểu sâu và kiểm chứng kiến thức một cách trực quan.

1.2 Mục tiêu đề tài

Đề tài tập trung xây dựng một môi trường giả lập (Simulator) có tính tương tác cao với các mục tiêu chính sau:

- **Tính tương tác:** Người dùng có thể tự do tạo, chỉnh sửa đồ thị bằng chuột (thêm/sửa/xóa nút và cạnh, thay đổi trọng số).
- **Tính trực quan:** Minh họa trực tiếp quá trình thuật toán “quét” qua các nút bằng hiệu ứng màu sắc, hiển thị sự thay đổi nhãn khoảng cách theo thời gian thực.
- **Tính giáo dục:** Cung cấp bảng nhật ký (log) chi tiết giải thích lý do chọn từng nút, cách cập nhật nhãn và quá trình hình thành đường đi ngắn nhất.

2 CƠ SỞ LÝ THUYẾT

2.1 Bản chất của thuật toán Dijkstra

Thuật toán Dijkstra là phương pháp tham lam (greedy) hiệu quả để tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh khác trong đồ thị có trọng số không âm.

Thuật toán hoạt động theo nguyên tắc “loại trừ dần dần”: bắt đầu từ đỉnh nguồn, liên tục chọn đỉnh có khoảng cách tạm thời nhỏ nhất để chốt nhãn vĩnh viễn, sau đó cập nhật khoảng cách đến các đỉnh lân cận thông qua phép thư giãn (relaxation). Điểm quan trọng của Dijkstra là khi một đỉnh được chốt nhãn vĩnh viễn, khoảng cách đến đỉnh đó là ngắn nhất tuyệt đối tại thời điểm đó.

2.2 Mã giả (Pseudocode) và Quy trình gán nhãn

Hệ thống sử dụng bộ nhãn $L = [d, p]$ cho mỗi đỉnh, trong đó:

- d (**Distance**): Khoảng cách ngắn nhất tạm tính từ đỉnh nguồn đến đỉnh hiện tại.
- p (**Parent**): Đỉnh cha (điểm nằm ngay trước) trên đường đi ngắn nhất.

Để hiện thực hóa lý thuyết trên, mã giả cốt lõi của thuật toán được trình bày như sau:

Mã giả thuật toán Dijkstra

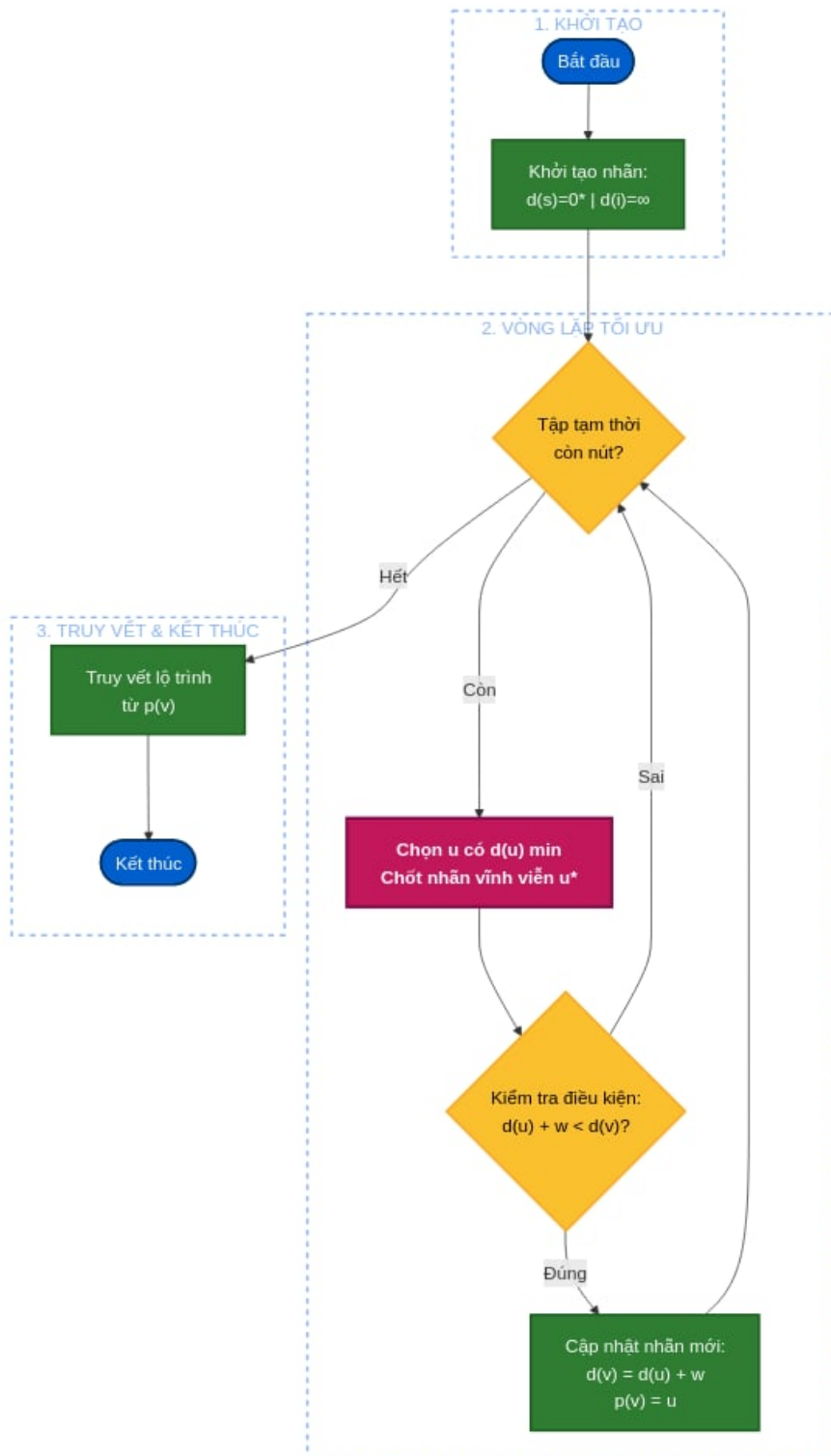
Input: Đồ thị $G = (V, E)$, đỉnh nguồn S

Output: Mảng khoảng cách d , mảng truy vết p

1. Khởi tạo $d[v] = \infty, p[v] = \text{null}$ với mọi đỉnh $v \in V$
2. Gán $d[S] = 0$
3. Khởi tạo tập Q chứa tất cả các đỉnh chưa được duyệt
4. **While** Q khác rỗng:
 - Chọn đỉnh $u \in Q$ có giá trị $d[u]$ nhỏ nhất (Chốt nhãn vĩnh viễn)
 - Xóa u khỏi tập Q
 - **For** mỗi đỉnh v kề với u (và $v \in Q$):
 - $alt = d[u] + \text{weight}(u, v)$
 - **If** $alt < d[v]$: // Quá trình nới lỏng (Relaxation)
 - * $d[v] = alt$
 - * $p[v] = u$

2.3 Lưu đồ giải thuật

Dưới đây là sơ đồ mô tả quy trình hệ thống tiếp nhận dữ liệu và thực hiện các bước lặp của thuật toán:



Hình 1: Lưu đồ các bước thực hiện thuật toán trong ứng dụng

2.4 Đánh giá độ phức tạp (Complexity Analysis)

Việc phân tích độ phức tạp giúp đánh giá khả năng mở rộng của thuật toán khi số lượng nút và cạnh trong đồ thị tăng lên:

- **Độ phức tạp thời gian (Time Complexity):** Trong bản hiện thực của ứng dụng (file `script.js`), thuật toán sử dụng vòng lặp duyệt qua tập hợp các đỉnh chưa thăm (`unvisited Set`) để tìm đỉnh có khoảng cách nhỏ nhất, mất $O(|V|)$ thời gian. Thao tác này lặp lại $|V|$ lần. Quá trình nói lỏng duyệt qua tất cả các cạnh mất $O(|E|)$. Tổng độ phức tạp thời gian là $\mathcal{O}(|V|^2 + |E|)$. Đối với đồ thị thưa trong thực tế, nếu cải tiến sử dụng Hàng đợi ưu tiên (Priority Queue), độ phức tạp có thể giảm xuống mức tối ưu là $\mathcal{O}(|E| + |V| \log |V|)$.
- **Độ phức tạp không gian (Space Complexity):** Hệ thống sử dụng đối tượng (Object) lưu trữ mảng danh sách kề (`adj`), mảng khoảng cách (`dist`) và mảng truy vết (`prev`). Tổng dung lượng bộ nhớ tỉ lệ thuận với số đỉnh và số cạnh, cho độ phức tạp không gian là $\mathcal{O}(|V| + |E|)$.

3 THIẾT KẾ VÀ HIỆN THỰC

3.1 Lựa chọn công nghệ

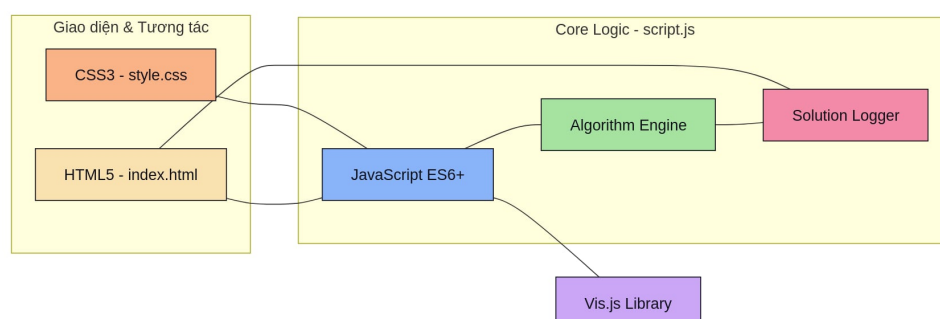
Để đảm bảo tính trực quan, tương tác và hiệu suất cao, các công nghệ sau được lựa chọn:

- **HTML5 & CSS3 (Catppuccin Dark Theme):** Xây dựng giao diện hiện đại, dễ nhìn, giảm mỏi mắt cho người dùng khi quan sát thời gian dài.
- **JavaScript (ES6+):** Xử lý toàn bộ logic thuật toán cấu trúc dữ liệu đồ thị ngay ở phía client (trình duyệt), đảm bảo tốc độ phản hồi tức thời không cần giao tiếp server.
- **Vis.js Network:** Thư viện chuyên dụng hỗ trợ việc render canvas, vẽ và tương tác vật lý với đồ thị (kéo thả nút, zoom, tùy chỉnh màu sắc động).

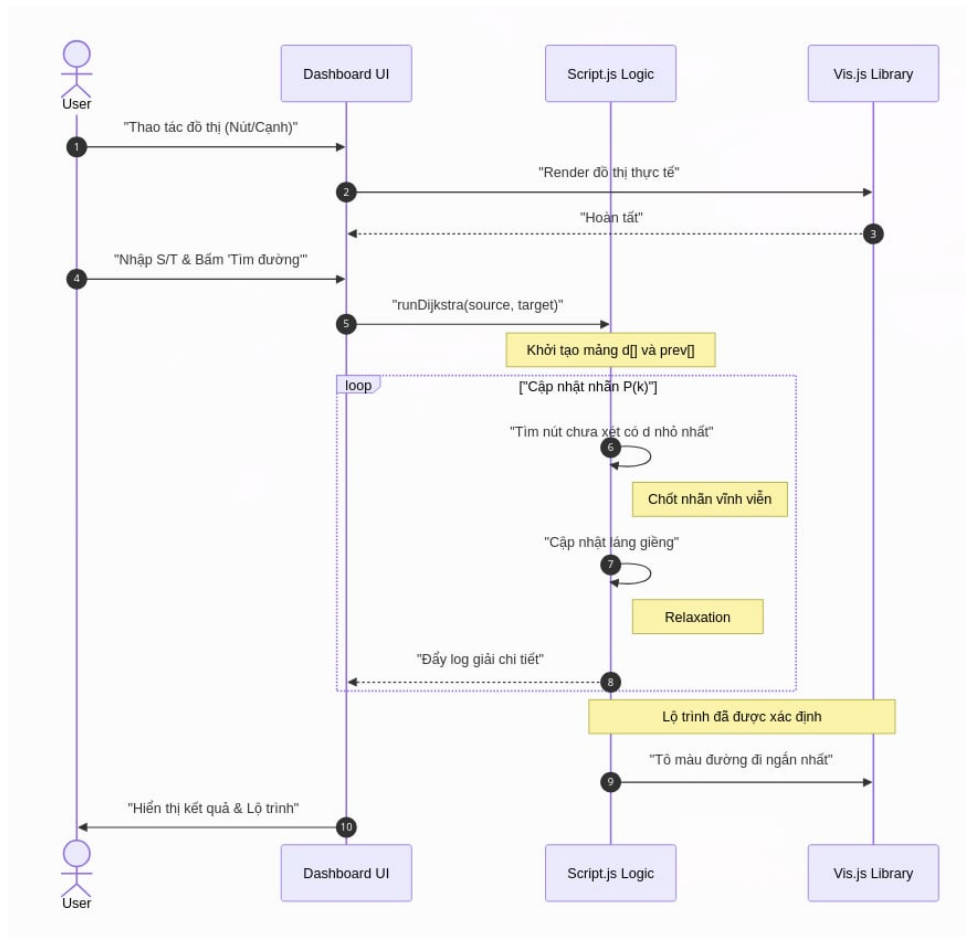
3.2 Cấu trúc dữ liệu và Vận hành ứng dụng

Đồ thị từ giao diện người dùng được mô hình hóa vào bộ nhớ JavaScript thông qua cấu trúc **Danh sách kề (Adjacency List)**. Khi người dùng bấm “Tìm đường đi”, hệ thống xây dựng một đối tượng `adj` duyệt qua `edges.get()` của Vis.js để thiết lập các liên kết và trọng số. Hệ thống được thiết kế theo kiến trúc phân tầng:

- **Algorithm Engine:** Xử lý logic cốt lõi của Dijkstra.
- **Solution Logger:** Phụ trách định dạng đầu ra thành các bước học thuật (log), chuyển đổi trạng thái số liệu thành chuỗi hiển thị dưới dạng mảng $L = [d, p]$.
- **Visualizer:** Quản lý hiển thị trên canvas, highlight đường đi ngắn nhất.



Hình 2: Mô hình kiến trúc phân lớp của simulator

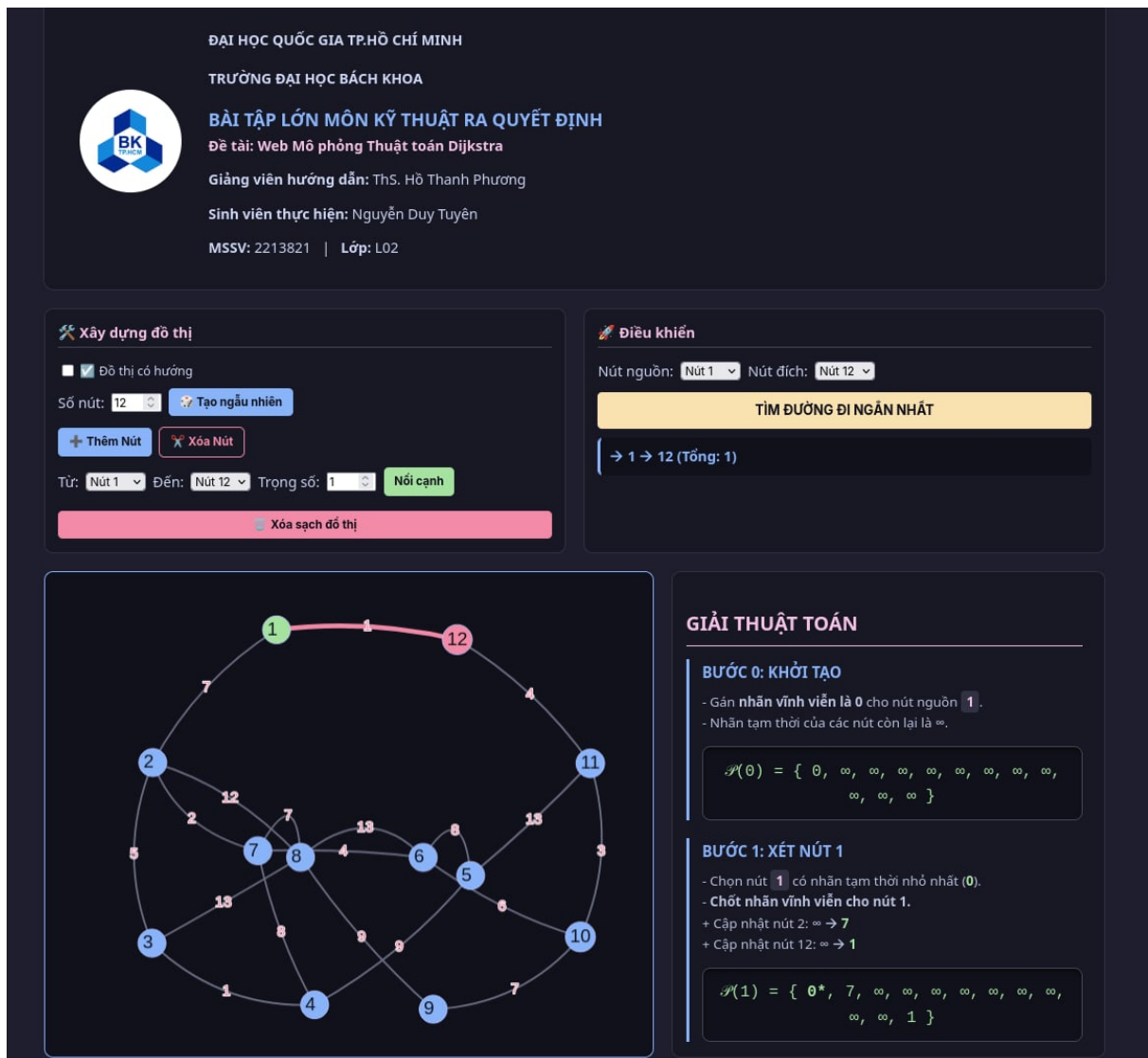


Hình 3: Trình tự xử lý dữ liệu từ lúc bấm nút đến lúc hiện kết quả

4 KẾT QUẢ VÀ THẢO LUẬN

4.1 Trải nghiệm người dùng trên Dashboard

Ứng dụng cho phép người dùng đóng vai trò là người thiết kế đồ thị. Các thao tác thêm nút, nối cạnh, chỉnh sửa trọng số diễn ra trực quan và tự nhiên. Tính năng “Tạo đồ thị ngẫu nhiên” giúp người dùng nhanh chóng có dữ liệu thử nghiệm mà không tốn nhiều thời gian.



Hình 4: Toàn cảnh giao diện mô phỏng với đồ thị mẫu

4.2 Đánh giá tính hiệu quả

Qua quá trình phát triển và thử nghiệm, ứng dụng đã đạt được mục tiêu giáo dục đề ra:

- **Màu sắc thông minh:** Đường đi ngắn nhất được highlight bằng màu hồng nổi bật, các nút đã thăm được đổi màu, dễ dàng phân biệt với các đỉnh chưa xét.
- **Nhật ký chi tiết:** Bảng log bên phải ghi nhận từng bước một cách rõ ràng. Hệ thống in ra chính xác phương trình truy vết (Ví dụ: $Nhân(j) + d(j,n) = Nhân(n)$), giúp người học đối chiếu trực tiếp với bài tập giải tay.

GIẢI THUẬT TOÁN

BƯỚC 0: KHỞI TẠO

- Gán **nhãn vĩnh viễn là 0** cho nút nguồn **1**.
- Nhãn tạm thời của các nút còn lại là ∞ .

$$\mathcal{P}(0) = \{ 0, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty \}$$

BƯỚC 1: XÉT NÚT 1

- Chọn nút **1** có nhãn tạm thời nhỏ nhất (**0**).
- **Chốt nhãn vĩnh viễn cho nút 1.**
- + Cập nhật nút 2: $\infty \rightarrow 7$
- + Cập nhật nút 12: $\infty \rightarrow 1$

$$\mathcal{P}(1) = \{ 0^*, 7, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, 1 \}$$

Hình 5: Phân tích kết quả và bảng nhật ký giải thuật

BƯỚC 2: XÉT NÚT 12

- Chọn nút **12** có nhãn tạm thời nhỏ nhất (1).
- **Chốt nhãn vĩnh viễn cho nút 12.**
- + Cập nhật nút 11: $\infty \rightarrow 5$

$$\mathcal{P}(2) = \{ 0^*, 7, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, 5, 1^* \}$$

TRUY VẾT LỘ TRÌNH

- Bắt đầu từ nút đích **12** lùi về nút nguồn **1**.
- **Quy tắc:** Tìm nút j đã chốt nhãn sao cho: $\text{Nhãn}(j) + d(j,n) = \text{Nhãn}(n)$.

→ Tại nút **12** ($L=1$): Nút đứng trước là **1** vì:
 $\text{Nhãn}(1) + d(1,12) = 0 + 1 = 1$ (Khớp!).

Hình 6: Phân tích truy vết lộ trình hiển thị trên hệ thống

4.3 Xử lý các trường hợp ngoại lệ (Edge Cases)

Để đảm bảo tính bền vững (robustness), hệ thống đã được lập trình để xử lý chặt chẽ các trường hợp dị thường trong lý thuyết đồ thị:

- **Đồ thị không liên thông:** Nếu chọn đỉnh nguồn và đỉnh đích thuộc hai thành phần liên thông tách biệt, vòng lặp Dijkstra sẽ tự động ngắt khi đỉnh tạm thời có khoảng cách nhỏ nhất vẫn là ∞ . Hệ thống sẽ in ra kết luận: *“Không tìm thấy đường đi”*.
- **Trọng số âm:** Do bản chất Dijkstra không xử lý được chu trình âm, giao diện đầu vào (ô input `edgeWeight`) đã được tích hợp ràng buộc `min="0"`, ngăn chặn ngay từ UI việc người dùng vô tình phá vỡ nguyên lý của thuật toán.

5 KẾT LUẬN

Hệ thống web mô phỏng trực quan thuật toán Dijkstra đã hoàn thành các mục tiêu đề ra. Ứng dụng không chỉ thực hiện đúng chức năng tính toán đường đi ngắn nhất mà còn là một công cụ hỗ trợ học tập hiệu quả, giúp người dùng hiểu sâu bản chất thuật toán thông qua trải nghiệm tương tác trực quan và bảng log học thuật chi tiết.

Trong tương lai, em dự định nâng cấp cấu trúc dữ liệu lên Priority Queue để tối ưu hiệu suất cho đồ thị cực lớn, đồng thời mở rộng hệ thống để hỗ trợ thêm các thuật toán tìm đường khác như Bellman-Ford (xử lý trọng số âm), A* Search và Floyd-Warshall, hướng tới xây dựng một nền tảng học tập thuật toán đồ thị toàn diện.

TÀI LIỆU THAM KHẢO

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, 1959.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
- [3] Phan Quốc Khánh, *Quy hoạch tuyến tính và các bài toán tối ưu trên đồ thị*, NXB Giáo dục, 2002.
- [4] MDN Web Docs, “JavaScript Reference and Web APIs,” 2026.
- [5] Vis.js Community, “Vis-network Documentation,” 2026.

PHỤ LỤC

LIÊN KẾT TRẢI NGHIỆM, MÃ NGUỒN VÀ VIDEO DEMO

1. Môi trường trải nghiệm trực tuyến (Live Demo):

Đường dẫn này đưa người dùng đến hệ thống mô phỏng đã được triển khai (deploy) trực tuyến trên nền tảng GitHub Pages. Tại đây, người dùng có thể tương tác trực tiếp, tự vẽ đồ thị, chỉnh sửa trọng số và chạy thử thuật toán Dijkstra mà không cần cài đặt bất kỳ phần mềm hay môi trường lập trình nào.



Quét mã QR để truy cập ứng dụng mô phỏng

2. Kho lưu trữ mã nguồn (Source Code):

Đây là kho lưu trữ (repository) toàn bộ mã nguồn của dự án trên GitHub, bao gồm các tệp tin giao diện (HTML/CSS), logic thuật toán xử lý đồ thị (JavaScript) và các thư viện liên quan. Việc công khai mã nguồn nhằm phục vụ mục đích đánh giá học thuật, cho phép người xem kiểm tra cấu trúc hệ thống, cách thức lưu trữ dữ liệu và tham khảo mã nguồn.



Quét mã QR để xem kiểm tra mã nguồn (GitHub)

3. Video thuyết trình:

Video này trình bày trực quan cách thức hoạt động của ứng dụng từ góc nhìn của người dùng thực tế. Nội dung video bao gồm các bước hướng dẫn thao tác tạo đồ thị, thêm/xóa nút, tùy chỉnh trọng số và mô tả trực tiếp quá trình thuật toán "quét" qua các đỉnh để tìm đường đi ngắn nhất.



Quét mã QR để xem Video hướng dẫn (YouTube)